Yet another Trading Strategy (YaTS) on US stocks

Abstract

This strategy implements a machine-learning strategy that transforms raw market and fundamental data into *Buy/Sell/Hold* signals and corresponding position sizes. The model outputs both the direction and strength of conviction, which are then mapped to trade sizes. Equity grows multiplicatively by applying daily returns to these positions, and profitability is assessed relative to a risk-free benchmark. Rolling-window validation ensures adaptation to changing regimes, with performance measured via both classification metrics (confusion matrix) and portfolio metrics (equity curve, Sharpe ratio, drawdown).

Overview

The workflow connects raw data, feature engineering, rolling-window classification, position sizing, and portfolio evaluation. Models are retrained sequentially, predictions are converted to trades, and the compounded equity curve shows whether the system consistently beats the risk-free rate.

Pseudocode

Step 0: Data Preparation

We label next-day returns and construct a multi-class classification target.

```
INPUT
  UNIVERSE := list of tickers
  DATE_RANGE := [t0, T]
  FEATURES := model columns except {ticker, Y}
  TARGET := direction ∈ {-1, 0, +1}

For each (permno, date):
  Y := next-day log return
  DIR := +1 if Y > τ_label
         -1 if Y < -τ_label
          0 otherwise
```

Mathematically:

$$Y_{i,t+1} = \ln \frac{P_{i,t+1}}{P_{i,t}}, \quad \mathrm{DIR}_{i,t} = \begin{cases} +1 & Y_{i,t+1} > \tau, \\ -1 & Y_{i,t+1} < -\tau, \\ 0 & \text{otherwise.} \end{cases}$$

Step 1: Rolling Windows

We simulate a live environment by training on expanding/sliding windows.

```
TIMESLICE := {initial=260d, horizon=60d, step=20d, fixedWindow=TRUE}
WINDOWS := []

while start + initial + horizon ≤ T:
    train_win := [start, start+initial)
    valid_win := [start+initial, start+initial+horizon)
    WINDOWS.append((train_win, valid_win))
    start := start + step
```

Step 2: Preprocessing

To stabilize features we apply per-date normalization.

```
function preprocess_slice(SLICE):
    for each date d:
        Xd := SLICE[*, d, FEATURES]
        Xd := winsorize(Xd, p=1%)
        Xd := zscore_by_date(Xd)
        SLICE[*, d, FEATURES] := Xd
    return SLICE
```

Step 3: Modeling

Classification predicts probabilities of up, down, or neutral.

```
MODEL_TYPE := logistic or tree-boost
HYPERGRID := parameter grid

for (train_win, valid_win) in WINDOWS:
    TR := preprocess_slice(DF[train_win])
    VA := preprocess_slice(DF[valid_win])

    Fit model on TR
    Predict class probabilities on VA
    Score by macro-F1
```

Each model estimates:

$$\hat{p}_{i,t}^{(+1)},\ \hat{p}_{i,t}^{(0)},\ \hat{p}_{i,t}^{(-1)},$$

where $\sum \hat{p}_{i,t} = 1$.

Step 4: Generating Signals and Trade Sizes

Probabilities are mapped to both direction and weight.

```
For each validation window:
    SCORE := p(+1) - p(-1)

    SIGNAL :=
        +1 if SCORE > τ_buy
        -1 if SCORE < τ_sell
         0 otherwise
```

```
    WEIGHT :=
        γ * SCORE      # scaled conviction
```

So the effective portfolio weight is:

$$w_{i,t} = \gamma \cdot \left(\hat{p}_{i,t}^{(+1)} - \hat{p}_{i,t}^{(-1)} \right),$$

clipped to $[-w_{\max}, +w_{\max}]$.

Step 5: Position Management

Positions persist until flipped or a maximum holding period is reached.

```
For each date t:
  if SIGNAL=+1: open/flip to long (w > 0)
  if SIGNAL=-1: open/flip to short (w < 0)
  if SIGNAL=0: hold if age < max_hold_days else exit
```

Step 6: Portfolio Equity Growth

We compute daily PnL, reinvested multiplicatively.

```
For each date t:
    port_ret[t+1] := Σ_i w_{i,t} * Y_{i,t+1}
    equity[t+1] := equity[t] * exp(port_ret[t+1])
```

Formally:

$$r_t = \sum_i w_{i,t} Y_{i,t+1}, \qquad E_{t+1} = E_t \cdot e^{r_t}.$$

Step 7: Evaluation

We report both classification accuracy and portfolio profitability.

```
Classification:
  Confusion matrix (pred_dir vs true_dir)
  Precision/Recall, Macro-F1
  Rank correlation (pred_score vs realized return)

Portfolio:
  Cumulative return
  Annualized return and volatility
  Sharpe ratio = (μ - r_f)/σ
  Max drawdown
  Equity curve plot
```

Confusion matrix:

$$C = \{c_{ij}\}, \quad c_{ij} = \#\{\text{true} = i, \text{pred} = j\}.$$

Sharpe ratio:

$$S = \frac{\mathbb{E}[r_t] - r_f}{\text{Std}(r_t)}.$$